



# Database Management Systems

## Additional Basic Operations

---

**Anchor:**

**Dr. Nagasundari S**

Department of Computer Science and Engineering

# Database Management Systems

---

## Unit 2: Advanced SQL in Database Design

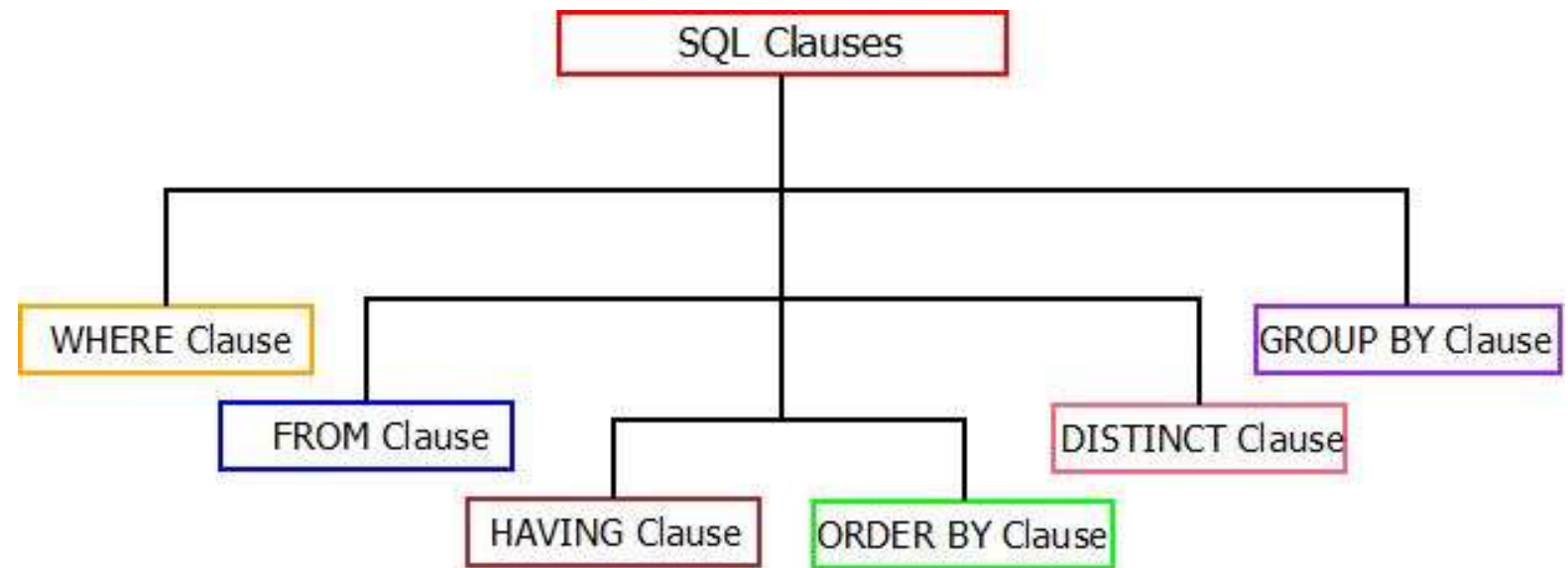
Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

**Dr. Nagasundari S**

Department of Computer Science and Engineering

### Additional Basic Operations

- Select clause
- Distinct clause
- Where clause
- From clause
- Order clause
- Having clause
- Group by clause
- String operations



```
SELECT <attribute and function list>  
FROM <table list>  
[ WHERE <condition> ]  
[ GROUP BY <grouping attribute(s)> ]  
[ HAVING <group condition> ]  
[ ORDER BY <attribute list> ];
```

- The **SELECT** clause lists the attributes desired in the result of a query.
- It corresponds to the projection operation of relational algebra.
- The **FROM** clause is used to specify the table name or the relation name.

**Example:** Find the names of all employees.

**Query:** select Fname, Lname from EMPLOYEE;

Output:

| Fname    | Lname   |
|----------|---------|
| John     | Smith   |
| John     | Smith   |
| Franklin | Wong    |
| Joyce    | English |
| Ramesh   | Narayan |
| James    | Borg    |
| Jennifer | Wallace |
| Ahmed    | Jabbar  |
| Alicia   | Zelaya  |

*NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)*

E.g., Name  $\equiv$  NAME  $\equiv$  name

Some people use upper case wherever we use bold font.

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **DISTINCT** after SELECT.

**Example:** Retrieve the name and address of all employees (display unique names).

**Query:** SELECT **distinct** Fname, Lname, Address  
FROM EMPLOYEE;

**Output:**

| Fname    | Lname   | Address                  |
|----------|---------|--------------------------|
| John     | Smith   | 731 Fondren,Houston,TX   |
| Franklin | Wong    | 638 voss,Houston,TX      |
| Joyce    | English | 5631 Rice,Houston,TX     |
| Ramesh   | Narayan | 975 Fire Oak, Humble, TX |
| James    | Borg    | 450 Stone, Houston,TX    |
| Jennifer | Wallace | 291 Berry, Bellaire,Tx   |
| Ahmed    | Jabbar  | 980 Dallas, Houston,TX   |
| Alicia   | Zelaya  | 3321 Castle, Spring,Tx   |

- The **SELECT** clause can contain arithmetic expressions involving the operations, **+**, **-**, **\***, **/** and operating on constants or attributes of tuples.

**Query:** select fname, salary/2 from EMPLOYEE;

**Table:**

| fname    | salary   |
|----------|----------|
| John     | 30000.00 |
| John     | 30000.00 |
| Franklin | 40000.00 |
| Joyce    | 25000.00 |
| Ramesh   | 38000.00 |

**Output:**

| fname    | salary/2     |
|----------|--------------|
| John     | 15000.000000 |
| John     | 15000.000000 |
| Franklin | 20000.000000 |
| Joyce    | 12500.000000 |
| Ramesh   | 19000.000000 |

- Query returns a relation that is same as the EMPLOYEE relation, except that the value of the attribute *salary* is divided by 2.
- Can rename “*salary/2*” using the **as** clause:  
select *fname*, *salary/2* as *monthly\_salary*

- The **WHERE** clause specifies conditions that the result must satisfy (row filter)
- It corresponds to the selection predicate of relational algebra.

Output:

**Example:** Find all employees in department no 5.

**Query:** select fname ,dno from EMPLOYEE where dno = 5;

| fname    | dno |
|----------|-----|
| John     | 5   |
| John     | 5   |
| Franklin | 5   |
| Joyce    | 5   |
| Ramesh   | 5   |

- SQL allows the use of logical connectives **AND**, **OR**, and **NOT**.
- The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.

Output:

- Comparisons can be applied to results of arithmetic expressions

**Example:** Find all employees in department no 5 with salary lesser than \$70,000.

**Query:** select fname, salary from employee where dno = 5 and salary < 70000;

| fname    | salary   |
|----------|----------|
| John     | 30000.00 |
| Franklin | 40000.00 |
| Joyce    | 25000.00 |
| Ramesh   | 38000.00 |

**Example:** For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

**Query:** SELECT Pnumber, Dnum, Lname, Address, Bdate  
FROM PROJECT, DEPARTMENT, EMPLOYEE  
WHERE Dnum = Dnumber AND Mgr\_ssn = Ssn AND Plocation = 'Stafford';

**Output:**

| <u>Pnumber</u> | <u>Dnum</u> | <u>Lname</u> | <u>Address</u>         | <u>Bdate</u> |
|----------------|-------------|--------------|------------------------|--------------|
| 10             | 4           | Wallace      | 291Berry, Bellaire, TX | 1941-06-20   |
| 30             | 4           | Wallace      | 291Berry, Bellaire, TX | 1941-06-20   |

- ORDER BY clause is a simple keyword that can be used with SQL aggregate functions, HAVING clause, or GROUP BY clause.
- The main function of the ORDER BY clause is to sort the result in either ascending or descending order.
- It means, sorting the result requires an ORDER BY clause.
- This can be done using either of the two keywords.
  - ASC – For sorting results in ascending order.
  - DESC – For sorting results in descending order.
- If we do not place either “ASC or DESC” at the end of the query, the data is sorted in ascending order by default.

**Example1:** List the employees based on descending order of their salary.

**Query:** select fname, lname, salary from EMPLOYEE order by salary desc;

**Example2:** List the employees based on ascending order of their salary

**Query:** select fname, lname, salary from EMPLOYEE order by salary asc;

**Output1:**

| fname    | lname   | salary   |
|----------|---------|----------|
| James    | Borg    | 55000.00 |
| Jennifer | Wallace | 43000.00 |
| Franklin | Wong    | 40000.00 |
| Ramesh   | Narayan | 38000.00 |
| John     | Smith   | 30000.00 |
| Joyce    | English | 25000.00 |
| Ahmed    | Jabbar  | 25000.00 |
| Alicia   | Zelaya  | 25000.00 |

| fname    | lname   | salary   |
|----------|---------|----------|
| Joyce    | English | 25000.00 |
| Ahmed    | Jabbar  | 25000.00 |
| Alicia   | Zelaya  | 25000.00 |
| John     | Smith   | 30000.00 |
| Ramesh   | Narayan | 38000.00 |
| Franklin | Wong    | 40000.00 |
| Jennifer | Wallace | 43000.00 |
| James    | Borg    | 55000.00 |

- SQL allows renaming relations and attributes using the **as** clause:

*old-name as new-name*

**Example:** Find the annual salary of employees.

**Query:** select fname,lname, salary\*12 as Annual\_Income from EMPLOYEE;

**Output:**

| fname    | lname   | Annual_Income |
|----------|---------|---------------|
| John     | Smith   | 360000.00     |
| Franklin | Wong    | 480000.00     |
| Joyce    | English | 300000.00     |
| Ramesh   | Narayan | 456000.00     |
| James    | Borg    | 660000.00     |
| Jennifer | Wallace | 516000.00     |
| Ahmed    | Jabbar  | 300000.00     |
| Alicia   | Zelaya  | 300000.00     |

**NOTE:** Keyword **as** is optional and may be omitted, e.g. instructor **as** T  $\equiv$  instructor T

- SQL includes a string-matching operator for comparisons on character strings.
- The operator **LIKE** uses patterns that are described using two special characters:
  - percent ( % ): The % character matches any substring.
  - underscore ( \_ ): The \_ character matches any character.

**Example:** Retrieve all employees whose address is in Houston, Texas.

**Query:** SELECT Fname, Lname FROM EMPLOYEE WHERE Address LIKE '%Houston,TX%';

**Output:**

| Fname    | Lname   | Address                |
|----------|---------|------------------------|
| John     | Smith   | 731 Fondren,Houston,TX |
| Franklin | Wong    | 638 voss,Houston,TX    |
| Joyce    | English | 5631 Rice,Houston,TX   |
| James    | Borg    | 450 Stone, Houston,TX  |
| Ahmed    | Jabbar  | 980 Dallas, Houston,TX |

*Note: Match the string “100%” like '100 \%' escape '\' in that above we use backslash (\) as an escape character*

**Examples:** Find all employees who were born during the 1960s.

**Query:** SELECT Fname, Lname, bdate FROM EMPLOYEE WHERE Bdate LIKE '196\_\_\_\_';

**Output:**

| Fname  | Lname   | bdate      |
|--------|---------|------------|
| John   | Smith   | 1965-01-09 |
| Ramesh | Narayan | 1962-09-15 |
| Ahmed  | Jabbar  | 1969-03-29 |
| Alicia | Zelaya  | 1968-01-19 |



# Database Management Systems

## Set Operations and Null Values

---

**Dr. Nagasundari S**

Department of Computer Science and Engineering

# Database Management Systems

## SET Operators in SQL

---



SQL Set operations are used to combine two or more SQL SELECT statements.

### **Types of Set Operations:**

1. Union
2. Union All
3. Intersect
4. Intersect All
5. Except
6. Except All

### 1. Union

In the UNION operation, the **number of columns and datatypes** must be the same in both the tables on which the UNION operation is being applied.

It **eliminates duplicate rows** from its result set.

**Syntax:**

```
SELECT column_name FROM table1
```

```
UNION
```

```
SELECT column_name FROM table2;
```

# Database Management Systems

## Example

Consider the relation: *student\_course*(name, course, grade) consisting of records of grades obtained by students in various courses.

The first image displays students and the grades obtained by them in the course 'Physics'

The second image displays students and the grades obtained by them in 'Mathematics'

Further on, these tables will be used to demonstrate the usage of set operations

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

# Database Management Systems

## Union

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
UNION  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

| grade |
|-------|
| S     |
| A     |
| B     |

Output :

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

## 2. Union All

The UNION ALL operation is similar to the UNION operation. It returns the set **without removing duplicates** and sorting the data. If a particular value is present  $c_1$  times in the first set(or relation) and  $c_2$  times in the second set(or relation), the number of duplicates or copies present in the result would be  **$c_1 + c_2$** .

### Syntax:

```
SELECT column_name FROM table1  
  
UNION ALL  
  
SELECT column_name FROM table2;
```

# Database Management Systems

## Union All

Query:

Output :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
UNION ALL  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

| grade |
|-------|
| S     |
| S     |
| A     |
| B     |
| B     |
| S     |
| S     |
| A     |
| A     |
| A     |

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

# Database Management Systems

## Union All



Query:

```
SELECT distinct grade FROM student_course  
WHERE course = 'Physics'  
  
UNION ALL  
  
SELECT distinct grade FROM student_course  
WHERE course = 'Mathematics';
```

| grade |
|-------|
| S     |
| A     |
| B     |
| S     |
| A     |

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

Notice the difference between the results of the two UNION ALL operations. This is due to the inclusion of the DISTINCT keyword. DISTINCT eliminates any duplicates and hence, the output of each SELECT statement has no duplicates. However, due to the usage of UNION ALL, duplicates are not eliminated in the result of the UNION ALL operation.

### 3. Intersect

It is used to combine two SELECT statements. The INTERSECT operation returns common rows from both the SELECT statements.

In the Intersect operation, the **number of columns and corresponding data types** must be the same.

It has **no duplicates**.

**Syntax:**

```
SELECT column_name FROM table1  
  
INTERSECT  
  
SELECT column_name FROM table2;
```

# Database Management Systems

## Intersect

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
INTERSECT  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

Output :

| grade |
|-------|
| S     |
| A     |

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

### 4. Intersect All

INTERSECT ALL is similar to the INTERSECT operation. It helps **retain duplicates**. If a particular value is present  $c_1$  times in the first set(or relation) and  $c_2$  times in the second set(or relation), the number of duplicates or copies present in the result would be  **$\min(c_1, c_2)$** .

#### Syntax:

```
SELECT column_name FROM table1  
  
INTERSECT ALL  
  
SELECT column_name FROM table2;
```

# Database Management Systems

## Intersect All

Query:

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
INTERSECT ALL  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

Output :

| grade |
|-------|
| S     |
| S     |
| A     |

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

### 5. Except

It combines the result of two SELECT statements. EXCEPT operator is used to display the rows that are present in the first query but absent in the second query.

It has **no duplicates**

**Syntax:**

```
SELECT column_name FROM table1
```

```
EXCEPT
```

```
SELECT column_name FROM table2;
```

# Database Management Systems

## Except

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'
```

EXCEPT

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics'
```

EXCEPT

```
SELECT grade FROM student_course  
WHERE course = 'Physics';
```

| grade |
|-------|
| B     |

| grade |
|-------|
|       |

Output :

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

(Empty Set)

### 6. Except All

EXCEPT ALL is similar to the EXCEPT operation. It helps **retain duplicates**. If a particular value is present  $c_1$  times in the first set(or relation) and  $c_2$  times in the second set(or relation), the number of duplicates or copies present in the result would be  **$\max(c_1 - c_2, 0)$**  i.e.  $c_1 - c_2$  if the difference is positive else 0 (the value itself will not be present)

#### Syntax:

```
SELECT column_name FROM table1
```

```
EXCEPT ALL
```

```
SELECT column_name FROM table2;
```

# Database Management Systems

## Except All

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'
```

EXCEPT ALL

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics'
```

EXCEPT ALL

```
SELECT grade FROM student_course  
WHERE course = 'Physics';
```

| grade |
|-------|
| B     |
| B     |

| grade |
|-------|
| A     |
| A     |

Output :

| name    | course  | grade |
|---------|---------|-------|
| Alex    | Physics | S     |
| Bert    | Physics | S     |
| Charles | Physics | A     |
| Dennis  | Physics | B     |
| Evans   | Physics | B     |

| name    | course      | grade |
|---------|-------------|-------|
| Alex    | Mathematics | S     |
| Bert    | Mathematics | S     |
| Charles | Mathematics | A     |
| Dennis  | Mathematics | A     |
| Evans   | Mathematics | A     |

# Database Management Systems

## Null values - Introduction

---



SQL has various rules for dealing with NULL values.

Generally, NULL has one of the three representations:

1. **Unknown value:** value exists but is not known, or it is not known whether or not the value exists. For example, when a person's date of birth is not known, it is represented as NULL in the database.
2. **Unavailable or withheld value:** value exists but is purposely withheld. For example, a person might not want to disclose his/her date of birth.
3. **Not applicable attributes:** the attribute does not apply to this tuple or is undefined for this tuple. For example, an attribute LastCollegeDegree would be NULL for a person who has no college degrees.

# Database Management Systems

## Null values - Introduction

---



- It is often not possible to determine the intended meaning of NULL. Hence, **SQL does not distinguish among the different meanings of NULL.**
- In general, each individual NULL value is considered to be **different** from every other NULL value in the various database records.
- Null values present special problems in relational operations, including arithmetic operations, comparison operations, and set operations.
- Result of an arithmetic operation ( +,-,\* or /) is **NULL** if **any of the input values are NULL.**

# Database Management Systems

## Null values in Arithmetic operations

Example:

```
SELECT Hours FROM WORKS_ON;
```

| Hours |
|-------|
| 32.5  |
| 7.5   |
| 10.0  |
| 10.0  |
| 10.0  |
| 10.0  |
| 20.0  |
| 20.0  |
| 40.0  |
| NULL  |
| 15.0  |
| 20.0  |

```
SELECT Hours+10 from WORKS_ON;
```

| Hours+10 |
|----------|
| 42.5     |
| 17.5     |
| 20.0     |
| 20.0     |
| 20.0     |
| 20.0     |
| 30.0     |
| 30.0     |
| 50.0     |
| NULL     |
| 25.0     |
| 30.0     |

The first image shows a part of the *Hours* column of the *WORKS\_ON* relation. The second image shows the result of adding 10 to each value in the *Hours* column. Notice how the **result of adding 10 to NULL results in NULL.**

- Comparisons involving NULL values pose a greater difficulty.
- To handle comparisons involving NULL values in SQL, a third logical value **UNKNOWN**, in addition to **TRUE** and **FALSE**, is used.
- For example, the expression **(1 < NULL)** would be evaluated as **UNKNOWN**. Similarly, **NOT(1 < NULL)** would be evaluated as **UNKNOWN** as well.
- In other words, any comparison operation involving NULL values would result in an UNKNOWN.

# Database Management Systems

## Three-valued logic truth table

Truth table for NOT, AND, OR operators:

|     |            |         |         |         |
|-----|------------|---------|---------|---------|
| (a) | <b>AND</b> | TRUE    | FALSE   | UNKNOWN |
|     | TRUE       | TRUE    | FALSE   | UNKNOWN |
|     | FALSE      | FALSE   | FALSE   | FALSE   |
|     | UNKNOWN    | UNKNOWN | FALSE   | UNKNOWN |
| (b) | <b>OR</b>  | TRUE    | FALSE   | UNKNOWN |
|     | TRUE       | TRUE    | TRUE    | TRUE    |
|     | FALSE      | TRUE    | FALSE   | UNKNOWN |
|     | UNKNOWN    | TRUE    | UNKNOWN | UNKNOWN |
| (c) | <b>NOT</b> |         |         |         |
|     | TRUE       | FALSE   |         |         |
|     | FALSE      | TRUE    |         |         |
|     | UNKNOWN    | UNKNOWN |         |         |

# Database Management Systems

## Is null and is not null

---



- If the **WHERE** predicate evaluates to **FALSE** or **UNKNOWN** for a tuple, the tuple is not added to the result.
- When a **JOIN** condition is specified, tuples with **NULL** values for the join attributes are not included in the result (unless it is an **OUTER JOIN**)
- SQL uses comparison operators **IS** and **IS NOT** to check if an attribute value is **NULL** or not, respectively

**Example:** Retrieve the names of all employees who do not have supervisors.

**Query:** `SELECT Fname, Lname FROM EMPLOYEE WHERE Super_ssn IS NULL;`

**Output:**

| Fname | Lname |
|-------|-------|
| James | Borg  |

# Database Management Systems

## Is unknown and is not unknown



- SQL allows us to test whether the result of a comparison is UNKNOWN, rather than TRUE or FALSE, by using the clauses **IS UNKNOWN** and **IS NOT UNKNOWN**.

**Example:** Retrieve the ssn of employees and their corresponding project numbers which satisfy the condition “Hours > 10 is unknown”.

**Query:** SELECT Essn, Pno FROM WORKS\_ON WHERE Hours > 10 IS UNKNOWN;

**Output:**

| Essn      | Pno |
|-----------|-----|
| 888665555 | 20  |

| Essn      | Pno | Hours |
|-----------|-----|-------|
| 123456789 | 1   | 32.5  |
| 123456789 | 2   | 7.5   |
| 333445555 | 2   | 10.0  |
| 333445555 | 3   | 10.0  |
| 333445555 | 10  | 10.0  |
| 333445555 | 20  | 10.0  |
| 453453453 | 1   | 20.0  |
| 453453453 | 2   | 20.0  |
| 666884444 | 3   | 40.0  |
| 888665555 | 20  | NULL  |
| 987654321 | 20  | 15.0  |
| 987654321 | 30  | 20.0  |

Only those tuples whose *Hours* attribute value is NULL will return UNKNOWN on performing the comparison.

# Database Management Systems

## Null values in distinct clause

---



- When a query uses the **SELECT DISTINCT** clause, duplicate tuples are eliminated.
- For this purpose, when comparing values of corresponding attributes from two tuples, the values are treated as **identical** if either **both are non NULL and equal in value**, or **both are NULL**.
- **For example, the tuples ('A', NULL) and ('A', NULL) are treated as identical. Hence, DISTINCT retains only one copy of such distinct tuples.**
- NOTE: Treatment of NULL above is different from the way NULL values are treated in predicates, where a comparison "NULL=NULL" would return UNKNOWN, rather than TRUE.
- The approach of treating tuples as identical if they have the same values for all attributes, even if some of the values are NULL, is also used for the set operations UNION, INTERSECTION, and EXCEPT.

1) When using the LIKE operator for string comparisons, which special character is used to match any single, individual character?

- A) %
- B) \*
- C) \_
- D) ?

2) Fill in the blank: The \_\_\_\_\_ operation is used to combine two SELECT statements into a single result set without removing any duplicate rows.

3) According to SQL rules, what is the result if any of the input values in an arithmetic operation (such as +, -, \*, or /) is a NULL value?

- A) UNKNOWN
- B) 0
- C) FALSE
- D) NULL

4) Fill in the blank: When using the ORDER BY clause to sort query results, if you do not explicitly place the ASC or DESC keywords at the end of the query, the data will be sorted in \_\_\_\_\_ order by default.

5) Assume a specific value appears 5 times in Table A and 3 times in Table B. If you execute a query using the EXCEPT ALL operation between Table A and Table B, how many times will that specific value appear in the final result set?

6) Because any direct mathematical comparison involving a NULL value evaluates to UNKNOWN, which specific operator must you use in a WHERE clause to successfully check if an attribute does not have a value?

- A) IS NULL
- B) = NULL
- C) IS UNKNOWN
- D) == NULL

### ANSWERS

- 1) C
- 2) UNION ALL
- 3) D
- 4) Ascending
- 5) 2
- 6) A



# THANK YOU

---

**Dr. Nagasundari S**

Department of Computer Science and Engineering

**[nagasundaris@pes.edu](mailto:nagasundaris@pes.edu)**